ESEMÉNYTÉR

Dokumentáció

EseményTér

Ahol minden program helyet kap

Készítette: Nagy Levéd Sámuel 13. I
Tóth Tamás 13. I

Tartalomjegyzék

1 Bevezetés ... 6
1.1. A feladat címe ... 6
1.2. Témaválasztás indoklása ... 6
1.3. Célkitűzés ... 6
1.4. Célközönség ... 6
2 Szoftver specifikáció ... 7
2.1. Megjelenés ... 7
2.1.1. Webes felület ... 7
2.2. Funkciók ... 9
2.2.1. Tanári funkciók ... 9
2.2.2. Diák funkciók ... 9
2.2.3. Admin funkciók ... 10
3 Fejlesztői specifikáció ... 11
3.1. Operációs rendszer ... 11
3.2. Felhasználandó programozási nyelvek ... 11
3.2.1. Frontend ... 11
3.2.2. Backend ... 11
3.3. Megoldás formátuma ... 11
3.4. Modulok ... 12
3.5. Szoftverfejlesztés ... 12
3.6. Tárolt adatok ... 12
3.6.1. Felhasználói adatok ... 12
3.6.2. Esemény adatok ... 12
4 Dokumentáció ... 13
4.1. Erőforrás terv, munkaidő nyilvántartás ... 13
4.2. Technikai dokumentáció ... 13
4.3. Forráskód dokumentáció ... 13
4.4. Felhasználói dokumentáció ... 13
5 Projekt adatlap ... 14
6 Algoritmusok ... 15
6.1. Bejelentkezés ... 15
6.2. Regisztráció ... 16

6.3. Email verifikáció ... 16
6.4. Kijelentkezés ... 17
6.5. Törzsadat keresés ... 17
6.6. Osztály kezelése ... 17
6.7. Kollab kérelem kezelése ... 18
6.8. Kollab esemény kérelmek listázása ... 18
6.9. Intézményi Kérelem Kezelése ... 19
6.10. Komment irás ... 19
6.11. Esemény létrehozás ... 20
6.12. Esemény Részvétel ... 21
6.13. Esemény listázása ... 22
6.14. Esemény priorizálás ... 23
6.15. Intézményhez csatlakozás ... 24
6.16. Ismétlődő esemény alkalom kezelése ... 25
7 Adatszerkezet ... 26
7.1. Adatbázis táblái és mezői ... 26
7.2. users (Felhasználók) ... 27
7.3. establishments (Iskolák) ... 27
7.4. classes (Osztályok) ... 28
7.5. staffs (Személyzet) ... 28
7.6. students (Diákok) ... 29
7.7. events (Események) ... 30
7.8. event_shows (Esemény megjelenítés) ... 31
7.9. event_messages (Esemény üzenetek) ... 31
7.10. event_requests (Esemény kérések) ... 31
7.11. polls (Szavazások) ... 32
7.12. poll_options (Szavazási lehetőségek) ... 32
7.13. poll_answers (Szavazatok) ... 32
7.14. regions (Megyék) ... 33
7.15. inner_regions (Járások) ... 33
7.16. settlements (Települések) ... 33
8 Tesztesetek ... 34
8.1. Frontend tesztelés ... 34

8.1.1. Sikeres regisztráció ... 34
8.1.2. Regisztráció gyenge jelszóval ... 35
8.1.3. Regisztráció ÁSZF elfogadása nélkül ... 36
8.1.4. Regisztráció eltérő jelszavakkal ... 37
8.1.5. Regisztráció hibás email formátummal ... 38
8.1.6. Sikeres bejelentkezés ... 39
8.1.7. Hibás bejelentkezés ... 40
8.1.8. Sikeres kijelentkezés ... 41
8.1.9. Diák profil beállítás sikeresen ... 42
8.1.10. Tanár profil beállítás sikeresen ... 43
8.1.11. Admin intézmény létrehozása sikeresen ... 44
8.1.12. Admin intézmény létrehozása duplikált névvel (hibakezelés) ... 45
8.2. Backend tesztelés ... 46
8.2.1. Regisztráció – sikeres ... 46
8.2.2. Login – hibás jelszó ... 47
8.2.3. Logout – token nélkül ... 48
8.2.4. Régió keresés ... 49
8.2.5. Intézmény létrehozás – sikeres ... 50
8.2.6. Osztály létrehozás – hiányzó név ... 51
8.2.7. Kérelmek kezelése – elfogadás ... 52
8.2.8. Kérelmek kezelése – nem admin felhasználó ... 53
8.2.9. Diáklista – nem létező intézmény ... 54
8.2.10. Esemény létrehozás – hibás dátumtartomány ... 55
9 Felhasználói dokumentáció ... 56
9.1. A rendszer rövid bemutatása ... 56
9.2. Belépés és fiókhasználat ... 56
9.2.1. Regisztráció ... 56
9.2.2. Email megerősítés ... 56
9.2.3. Bejelentkezés ... 56
9.2.4. Bejelentkezés szerepkör szerint (diák, tanár, admin) ... 57
9.3. Navigáció és fő oldalak ... 58
9.4. Események kezelése felhasználóként ... 58
9.4.1. Eseménylista ... 58

9.4.2. Esemény részletek ... 58
9.4.3. Részvételi visszajelzés ... 58
9.4.4. Kedvenc jelölés ... 58
9.4.5. Kommentelés ... 59
9.5. Eseménynaptár használata ... 59
9.6. Esemény létrehozása ... 59
9.7. Profilkezelés ... 59
9.8. Intézményvezetői funkciók ... 60
9.9. Függő jóváhagyás állapot ... 60
9.10. Jogszabályi tájékoztatók ... 60
9.11. Rövid felhasználói folyamatok ... 60
9.11.1. Diák ... 60
9.11.2. Tanár ... 60
9.11.3. Adminisztrátor ... 61

1 Bevezetés

1.1. A feladat címe

Eseménytér

1.2. Témaválasztás indoklása

Az iskolai életben rendszeresen szerveznek különféle programokat – például kirándulásokat, ünnepségeket, versenyeket vagy sportnapokat. Ezek szervezése és kommunikációja gyakran sok időt és adminisztrációt igényel.

A hagyományos módszerek (szóbeli bejelentés, papíros lista, közösségi csoportok) könnyen átláthatatlanná válnak, az információk elveszhetnek, és a kommunikáció hiányos lehet.

1.3. Célkitűzés

A projekt célja egy olyan webes rendszer létrehozása, amely központi felületen kezeli az iskolai eseményeket.

A rendszer lehetővé teszi:

- események létrehozását és kezelését
- jelentkezések nyomon követését
- szavazások lebonyolítását
- kommunikációt (kommentek, értesítések)

A rendszer több intézmény kezelésére is alkalmas, így skálázható megoldást biztosít.

1.4. Célközönség

A rendszer elsősorban az iskolai közösség számára készült, mivel náluk jelentkezik leginkább az az igény, hogy az események szervezése és kommunikációja átlátható, gyors és mindenki számára elérhető legyen akár több intézmény számára.

2 Szoftver specifikáció

2.1. Megjelenés

2.1.1. Webes felület

A rendszer egy reszponzív, egyoldalas webalkalmazás (SPA), amely asztali és mobil böngészőben egyaránt használható.

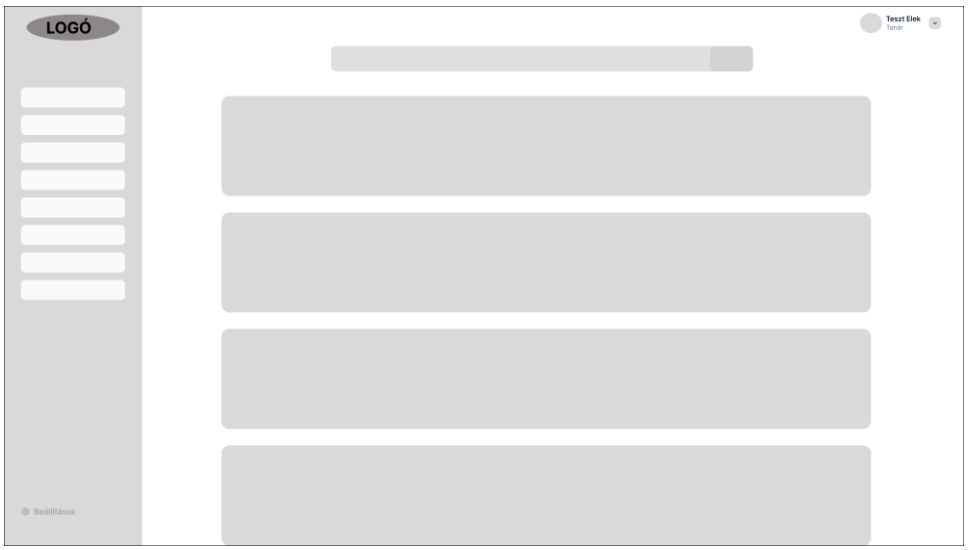
Főbb felületek:

- Bejelentkezés / Regisztráció
- Főoldal (eseménylista)
- Naptár nézet (calendar view – eseménylista naptári megjelenítéssel)
- Esemény részletező oldal
- Esemény létrehozása / szerkesztése
- Profil

Jellemzők:

- letisztult UI
- könnyű navigáció
- gyors keresés és szűrés

Események felkeresésére szolgáló oldal koncepció



Az események felkeresésére szolgáló oldal átlátható eseménylistát és keresőfelületet kínál a felhasználók számára, lehetővé téve, hogy gyorsan megtalálják a számukra érdekes eseményeket.

Események feltöltéséhez szolgáló oldal koncepció

Az események feltöltésére szolgáló oldal egy letisztult, egyszerű űrlapot kínál, ahol a szervezők könnyedén megadhatják az esemény részleteit, például címet, időpontot és leírást, valamint feltölthetik az eseményhez kapcsolódó képeket vagy dokumentumokat.

A weboldal reszponzív kialakítású, amely biztosítja, hogy az oldal mobilon és tableten egyaránt jól használható legyen.

2.2. Funkciók

2.2.1. Tanári funkciók

- Események létrehozása (név, időpont, helyszín, leírás)
- Ismétlődő események létrehozása (heti ismétlődés beállítása)
- Ismétlődő alkalmak kezelése (átütemezés, elmaradás jelölése)
- Jelentkezési feltételek beállítása (osztály, évfolyam, iskola)
- Résztvevők listájának kezelése
- Szavazások létrehozása eseményekhez
- Kommentek moderálása
- Esemény állapot kezelése (aktív / lezárt)

2.2.2. Diák funkciók

- Események listázása és részleteinek megtekintése
- Jelentkezés eseményre
- Ismétlődő eseménynek való jelentkezés (az egész sorozatra)
- Jelentkezés visszavonása
- Részvétel összesítése (pl. 20 felhasználó jelentkezett)
- Szavazásokon való részvétel, eredmények követése
- Diákok hozzászólhatnak eseményekhez
- Hozzászólások írása eseményekhez
- „Kedvenc esemény" funkció (jelölés/kijelölés)

2.2.3. Admin funkciók

- Teljes körű hozzáférés minden funkcióhoz és modulhoz
- Iskolák regisztrációjának jóváhagyása vagy törlése
- Felhasználók kezelése (létrehozás, törlés, jogosultság)
- Események kezelése: bármely esemény szerkesztése, törlése, lezárása
- Szavazások kezelése
- Kommentek moderálása
- Statisztikák megtekintése
- Rendszerbeállítások módosítása
- Adatbázis mentések kezelése

3 Fejlesztői specifikáció

3.1. Operációs rendszer

Windows 10, Windows 11

3.2. Felhasználandó programozási nyelvek

3.2.1. Frontend

- Vue 3
- Vite
- Vue Router
- Vuex
- Axios
- Bootstrap 5
- Playwright (E2E tesztek)

3.2.2. Backend

- Laravel 12
- PHP 8.2+
- Laravel Sanctum
- Eloquent ORM
- PHPUnit
- MySQL (adatbázis)

3.3. Megoldás formátuma

A rendszer modulokból álló alkalmazás, elkülönített Frontend és Backend komponensekkel

3.4. Modulok

- Felhasználó és authentikációs modul
- Intézmény és tagságkezelő modul
- Osztálykezelő modul
- Eseménykezelő modul
- Részvételi visszajelzés modul
- Kérés / szűrés / listázás modul
- Komment és kedvenc modul
- Naptári nézet modul
- Kérlelem / jóváhagyási modul

3.5. Szoftverfejlesztés

A fejlesztést kisebb szakaszokra bontva végezzük. Első lépésként az alapvető funkciók kerülnek megvalósításra, mint például a felhasználói bejelentkezés és az események alapvető kezelése. Ezek után kerülnek sorra a kiegészítő funkciók, mint a szavazások, az értesítések vagy a több iskola egyidejű kezelése.

3.6. Tárolt adatok

3.6.1. Felhasználói adatok

- Diákok: név, e-mail cím, osztály, évfolyam, jelszó (titkosított formában)
- Tanárok: név, e-mail cím, tantárgy (opcionális), jogosultságok
- Adminisztrátorok: név, e-mail cím, teljes jogosultsági szint

3.6.2. Esemény adatok

- Esemény alapadatok: cím, leírás, kezdés/befejezés időpontja
- Ismétlődési információ: ismétlődés típusa, befejezési dátuma, szülőeseményre való hivatkozás
- Állapot információ: aktív/lezárt/elmaradt státusz
- Részvételi adatok: igen vagy nem
- Kommentek és kedvencek információja

4 Dokumentáció

4.1. Erőforrás terv, munkaidő nyilvántartás

A projekt elkészítéséhez leginkább a saját időnkre és munkánkra van szükség. A feladatokat egymás között felosztjuk: egyikünk elsősorban a frontend fejlesztéssel foglalkozik (felhasználói felület kialakítása, reszponzív megjelenés, kezelhetőség), míg másikunk a backend oldalt készíti (adatbázis tervezés, üzleti logika, API-fejlesztés). A munkát kisebb szakaszokra bontjuk, így pontosan követhető, hogy hol tartunk a projektben. Ez biztosítja az átláthatóságot, és segít abban, hogy minden részidőben elkészüljön.

4.2. Technikai dokumentáció

Ebben a részben bemutatjuk, hogyan épül fel a rendszer két fő része:

- **Frontend**: a felhasználói felület, amelyen keresztül a diákok, tanárok és adminok használják a rendszert. Ide tartozik a megjelenés, a bejelentkezési és regisztrációs oldalak, az események listázása, valamint az interaktív funkciók (pl. szavazás, kommentelés).
- **Backend**: a háttérben futó rendszer, amely felelős az adatok kezeléséért és tárolásáért. Ide tartozik az adatbázis, az események és felhasználói adatok mentése, valamint a jogosultságok és értesítések kezelése.

A technikai dokumentáció célja, hogy érthetően bemutassa, hogyan kapcsolódik össze a frontend a backenddel, és hogyan valósul meg az adatok tárolása és feldolgozása. A dokumentáció tartalmazza az idegen szavak, rövidítések magyarázatát is, hogy nem szakmai olvasók számára is feldolgozható legyen.

4.3. Forráskód dokumentáció

A forráskódot úgy írjuk meg, hogy frontend és backend oldalon egyaránt könnyen átlátható legyen. A változóknak és függvényeknek beszédes nevet adunk, és minden fontosabb résznél rövid magyarázatot (kommentet) helyezünk el.

4.4. Felhasználói dokumentáció

A felhasználói dokumentáció bemutatja a rendszer használatát a különböző szerepkörök számára (tanár, diák, adminisztrátor). Tartalmazza a bejelentkezés, események megtekintésének és létrehozásának, valamint a szavazások és értesítések használatának lépéseit. Célja, hogy a felhasználók könnyen eligazodjanak a felületen és megértsék az egyes funkciók működését technikai ismeretek nélkül.

5 Projekt adatlap

Projekt neve: EseményTér

Feladat rövid ismertetése: Az EseményTér egy olyan webes rendszer, amely átláthatóbbá és egyszerűbbé teszi az iskolai programok szervezését és kommunikációját tanárok és a diákok számára.

Felhasznált programozási nyelvek: Vue3, Vite, Vue Router, Axios, Vuex, MySQL, PHP, JavaScript

Készítők: Nagy Levéd Sámuel, Tóth Tamás

6 Algoritmusok

6.1. Bejelentkezés

```
ALGORITMUS: Bejelentkezés

BEMENET:
-  email
-  password


LÉPÉSEK:
1. Bemenet ellenőrzése:
      o  email kötelező, string, létező felhasználói email
      o  password kötelező, string
2. Felhasználó lekérése email alapján
3. Ha nincs ilyen felhasználó, vagy jelszó hash ellenőrzés hibás:
      o  401, „Hibás jelszó vagy email”
4. Sanctum token létrehozása
5. Válasz:
      o  user (id, name, email, establishment_id)
      o  token
      o  siker üzenet


ALGORITMUS VÉGE
```

6.2. Regisztráció

```
ALGORITMUS: Regisztráció

BEMENET:
-  username
-  email
-  password
-  password_confirmation


LÉPÉSEK:
1. email kisbetűsítése
2. Validáció:
      o  username required
      o  email required, email, unique
      o  password required, confirmed, min 6, mixed case + number
3. Új felhasználó létrehozása
      o  name <- username
      o  email <- email
      o  password <- hash(password)
4. Email verifikációs értesítés küldése
5. Sanctum token generálása
6. Válasz: user + token + email_verified=false


ALGORITMUS VÉGE
```

6.3. Email verifikáció

```
ALGORITMUS: EmailVerifikáció


RÉSZFOLYAMATOK:
1. Verify link:
      o  ha már verified, visszajelzés
      o  különben markEmailAsVerified + Verified event
2. Újraküldés:
      o  nem verified felhasználónak új verifikációs email küldése
3. Státusz:
      o  verified boolean visszaadása


ALGORITMUS VÉGE
```

6.4. Kijelentkezés

```
ALGORITMUS: Kijelentkezés


LÉPÉSEK:
1. Hitelesített felhasználó aktuális tokenjének lekérése
2. Aktuális access token törlése
3. Válasz: "Sikeres kijelentkezés"


ALGORITMUS VÉGE
```

6.5. Törzsadat keresés

```
ALGORITMUS: TörzsadatKeresés


RÉSZFOLYAMATOK:
1. Régiók listázása és keresése
2. Járások listázása és keresése (region_id szűrővel)
3. Települések listázása és keresése (inner_region_id szűrővel)
4. Intézmények listázása:
      o  settlement_id vagy region_id alapú szűrés
      o  opcionális title search
5. Saját intézmények és intézmény részletek lekérdezése


ALGORITMUS VÉGE
```

6.6. Osztály kezelése

```
ALGORITMUS: OsztályKezelés


RÉSZFOLYAMATOK:
1. Osztály létrehozás
2. Osztályok listázása
3. Osztálytagok listázása
4. Osztályfőnök módosítása
5. Diákok hozzáadása és eltávolítása

FŐ SZABÁLYOK:
-  létrehozás/listázás/tanár módosítás admin vagy staff szerepkör szerint
   védett
-  diák mozgás csak admin jogosultsággal
-  intézmény és osztály összerendelés ellenőrzése kötelező


ALGORITMUS VÉGE
```

6.7. Kollab kérelem kezelése

```
ALGORITMUS: KollabKérelemKezelése

BEMENET:
-  establishmentId
-  event_id
-  action: accept vagy reject
-  users[] (action=accept esetén kötelező)


LÉPÉSEK:
1. Hitelesített felhasználó és admin jogosultság ellenőrzése
2. event_id és action validálása
3. Ha action=accept, users[] lista validálása (létező, egyedi felhasználó
   azonosítok)
4. Tranzakció:
      o  EventRequest törlése az adott intézményre
      o  ha accept: intézményhez tartozó felhasználók elemeknek event_shown
         létrehozás
      o  ha a felhasználóknak más intézményi event_shown sorban már kedvenc
         az esemény, az új sor is_favourite=true érteket kap
5. Válasz: „Művelet végrehajtva”


ALGORITMUS VÉGE
```

6.8. Kollab esemény kérelmek listázása

```
ALGORITMUS: KollabEseményKérelmekListázása

BEMENET:
-  establishmentId


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. Admin jogosultság ellenőrzése az intézményben
3. EventRequest rekordokból event_id-k kigyűjtése
4. Események lekérése start_date szerint
5. creator_name mező kiegészítése
6. Válasz: eseménylista


ALGORITMUS VÉGE
```

6.9. Intézményi Kérelem Kezelése

```
ALGORITMUS: IntézményiKérelemKezelés

BEMENET:
-  establishment_id
-  action: accept vagy reject
-  request_id[]


LÉPÉSEK:
1. Input validáció
2. Admin jogosultság ellenőrzése
3. Csak az adott intézményhez tartozó kérelmek feldolgozása
4. Ha action = accept:
      o  teacher -> Staff rekord létrehozása
      o  student -> Student rekord létrehozása
      o  user.establishment_id frissítése
      o  request törlése
5. Ha action = reject:
      o  kérelmek törlése
6. Válasz: accepted/rejected darabszám


ALGORITMUS VÉGE
```

6.10. Komment irás

```
ALGORITMUS: KommentÍrás

MEGJEGYZÉS (JELENLEGI ÁLLAPOT):
-  A rendszerben van event_messages tábla és komment darabszám számítás
-  Külön komment létrehozó API endpoint jelenleg nincs a routes



ALGORITMUS VÉGE
```

6.11. Esemény létrehozás

```
ALGORITMUS: EseményLétrehozás

BEMENET (KÖZÖS):
-  type: local vagy global
-  establishment_id
-  target_group: osztaly_szintu vagy evfolyam_szintu vagy teljes_iskola
-  selected_class_ids[] (ha target_group=osztaly_szintu)
-  selected_grade_ids[] (ha target_group=evfolyam_szintu)
-  title, description, content
-  start_date, end_date, (end_date > start_date)
-  users[]

BEMENET (ISMÉTLŐDÉS, CSAK LOCAL ESETBEN):
-  is_recurring: true/false
-  recurrence_frequency: weekly (ha is_recurring=true)
-  recurrence_until (ha is_recurring=true)

BEMENET (GLOBÁL ESETBEN)
-  collab_establishment_ids[] (legalább 1)

LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. Bemenet validálása.
3. Ismétlődési szabály ellenőrzés
      o  ha is_recurring=true, recurrence_until nem lehet korábbi, mint
         start_date napja
      o  ismétlődő esemény csak local típussal engedélyezett
4. Célcsoport ellenőrzése:
      o  osztaly_szintu esetén minden selected_class_ids elemnek az
         intézményhez kell tartoznia
      o  evfolyam_szintu esetén minden selected_grade_ids elemnek szerepelnie
         kell az intézmény osztályai között
5. Ha type=local:
      o  csak staff hozhat létre saját intézményben
      o  tranzakcióban esemény rekord létrehozása (status=upcoming)
      o  event_shown rekordok létrehozása: készítő + intézményhez tartozó,
         kiválasztott felhasználók
      o  ha is_recurring=true: heti alkalmak generálása recurrence_until
         dátumig
6. Ha type=global:
      o  csak admin hozhat létre saját intézményben
      o  tranzakcióban esemény rekord létrehozása (ismétlődés kikapcsolva)
      o  event_shown rekordok létrehozása saját intézményhez tartozó
         felhasználók listára
      o  EventRequest rekordok létrehozása minden collab_establishment_ids
         elemre
7. Válasz: 201, „Esemény létrehozva” + esemény

   ALGORITMUS VÉGE
```

6.12. Esemény Részvétel

```
ALGORITMUS: EseményRészvétel

BEMENET:
-  eventId (path)
-  answer: y vagy n


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. answer mező validálása (csak y vagy n)
3. Esemény létezésének ellenőrzése
4. Jogosultság ellenőrzése: a felhasználó csak számára látható eseményre
   válaszolhat (event_shown)
5. event_deedbacks táblában updateOrInsert végrehajtás
6. Statisztikák újraszámolása:
      o  attending_count (y)
      o  not_attending_count (n)
7. Válasz: mentett válasz + statisztikák


ALGORITMUS VÉGE
```

6.13. Esemény listázása

```
ALGORITMUS: EseményListázás

BEMENET:
-  establishmentId


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése.
2. Intézményi tagság ellenőrzése.
3. Látható esemény id-k kigyűjtése event_shown-ból, user_id +
   establishment_id alapján
4. Esemény lekérdezése:
      o  azonnal látható események
      o  + olyan események, amelyek recurrence_parent_event_id mezőben egy
         látható szülőeseményre mutatnak

5. Rendezés start_date szerintnövekvő sorrendben
6. Kiegészítés:
      o  creator_name
      o  attending_count, not_attending_count
      o  participant_count
      o  user_participation
      o  comment_count
      o  is_favourite
7. Válasz: esemény tömb


ALGORITMUS VÉGE
```

6.14. Esemény priorizálás

```
ALGORITMUS: KedvencEsemény

BEMENET:
-  eventId (path)


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. Esemény létezés ellenőrzése
3. Jogosultság ellenőrzése:
      o  csak olyan esemény jelölhető kedvencnek, amelyre a felhasználónak
         van event_shown sora
4. Aktuális kedvenc állapot ellenőrzése (is_favourite)
5. Toggle művelet:
      o  ha jelenleg false, akkor true-ra állítás
      o  ha jelenleg true, akkor false-ra állítás
6. Mentés event_shown táblában (event_id + user_id kombináció(k) frissítése)
7. Válasz:
      o  "Esemény hozzáadva a kedvencekhez" vagy
      o  "Esemény eltávolítva a kedvencek közül"


ALGORITMUS VÉGE
```

6.15. Intézményhez csatlakozás

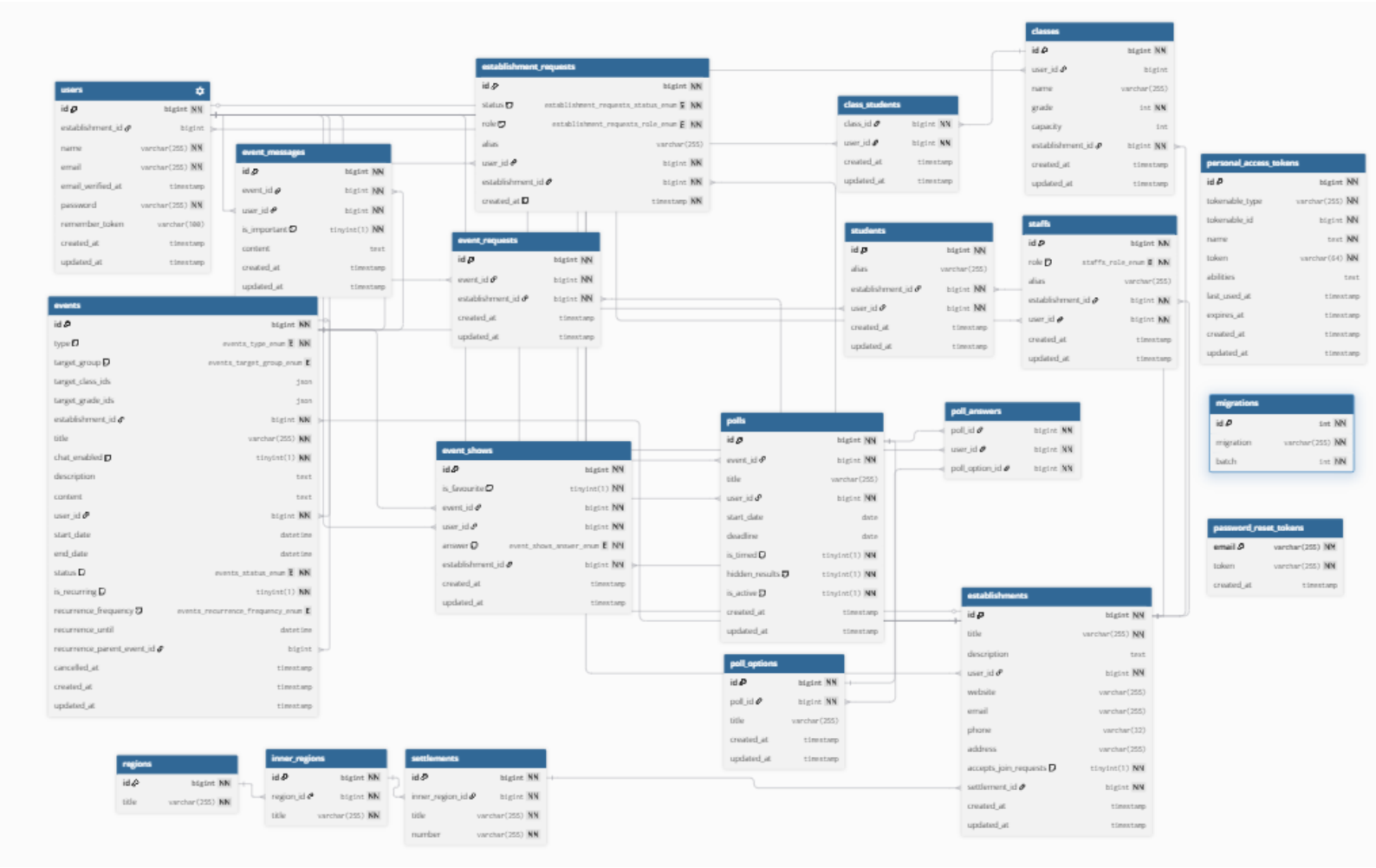
```
ALGORITMUS: IntézményhezCsatlakozás

BEMENET:
-  establishment_id
-  role: student vagy teacher


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. Ellenőrzés:
      o  van-e már kérelem ugyanarra az intézményre
      o  már tag-e staff/student táblák alapján
3. Validáció:
      o  establishment_id létezik
      o  role csak staff vagy student lehet
4. establishment_request rekord létrehozása
5. Válasz: 201, „Kérelem benyújtva”


ALGORITMUS VÉGE
```

6.16. Ismétlődő esemény alkalom kezelése

```
ALGORITMUS: IsmétlődőEseményAlkalomKezelése

BEMENET:
-  eventId (path)
-  action: reschedule vagy cancel
-  start_date (action=reschedule esetén kötelező)
-  end_date (action=reschedule esetén kötelező, end_date > start_date)


LÉPÉSEK:
1. Hitelesített felhasználó ellenőrzése
2. Esemény létezés ellenőrzése
3. Jogosultság ellenőrzése:
      o  csak az esemény létrehozója kezelheti az alkalmat
4. Bemenet validálása action alapján
5. Ha action=cancel:
      o  event.status <- ended
      o  event.cancelled_at <- now
      o  válasz: "Az alkalom elmaradtként jelölve"
6. Ha action=reschedule:
      o  event.start_date <- új start_date
      o  event.end_date <- új end_date
      o  event.status <- upcoming
      o  event.cancelled_at <- null
      o  válasz: "Az alkalom időpontja sikeresen módosítva" + esemény


ALGORITMUS VÉGE
```

7 Adatszerkezet

7.1. Adatbázis táblái és mezői



7.2. users (Felhasználók)

A rendszer összes regisztrált felhasználóját tartalmazza.
Fő mezők:

- **name, email, password**
- **email_verified_at:** e-mail megerősítés időpontja
- **establishment_id:** a felhasználóhoz tartozó intézmény
- **created_at, updated_at:** létrehozás és módosítás dátuma

users		
id	bigint	NN
establishment_id	bigint	
name	varchar(255)	NN
email	varchar(255)	NN
email_verified_at	timestamp	
password	varchar(255)	NN
remember_token	varchar(100)	
created_at	timestamp	
updated_at	timestamp	
name_updated_at	timestamp	

7.3. establishments (Iskolák)

Az iskolák adatait tárolja.
Fő mezők:

- **title:** iskola neve
- **description:** leírás
- **user_id:** az intézmény létrehozója / kezelője
- **website:** weboldal
- **email:** kapcsolattartási e-mail
- **phone:** telefonszám
- **address:** cím
- **accepts_join_requests:** kérelmek fogadása
- **settlement_id:** adott település hivatkozása
- **created_at, updated_at:** létrehozás és módosítás dátuma

establishments		
id	bigint	NN
title	varchar(255)	NN
description	text	
user_id	bigint	NN
website	varchar(255)	
email	varchar(255)	
phone	varchar(32)	
address	varchar(255)	
accepts_join_requests	tinyint(1)	NN
settlement_id	bigint	NN
created_at	timestamp	
updated_at	timestamp	

7.4. classes (Osztályok)

Az iskolákhoz tartozó osztályokat tárolja.
Fő mezők:

- **name:** osztály neve
- **grade:** évfolyam
- **capacity:** osztály létszámkerete
- **user_id:** az osztályhoz rendelt tanár
- **establishment_id:** kapcsolódó iskola
- **created_at, updated_at:** létrehozás és módosítás dátuma

classes	
id	bigint NN
user_id	bigint
name	varchar(255)
grade	int NN
capacity	int
establishment_id	bigint NN
created_at	timestamp
updated_at	timestamp

7.5. staffs (Személyzet)

Az intézményekhez tartozó tanári és adminisztratív felhasználókat tárolja.
Fő mezők:

- **role:** szerepkör (admin / teacher)
- **alias:** intézményi név
- **establishment_id:** kapcsolódó iskola
- **user_id:** felhasználó azonosító
- **class_id:** az adott osztályhoz rendelhető tanár
- **created_at, updated_at:** létrehozás és módosítás dátuma

staffs	
id	bigint NN
role	staffs_role_enum E NN
alias	varchar(255)
establishment_id	bigint NN
user_id	bigint NN
created_at	timestamp
updated_at	timestamp

7.6. students (Diákok)

A diákok intézményi és osztálybeli hozzárendelését tárolja.

Fő mezők:

- **establishment_id:** kapcsolódó iskola
- **class_id:** osztály
- **user_id:** felhasználó azonosító
- **alias:** megjelenített név / becenév
- **created_at, updated_at:** létrehozás és módosítás dátuma

students	
id	bigint NN
alias	varchar(255)
establishment_id	bigint NN
user_id	bigint NN
created_at	timestamp
updated_at	timestamp

7.7. events (Események)

A rendszer eseményeit tárolja.

Fő mezők:

- **type:** esemény típusa (local / global)
- **establishment_id:** kapcsolódó iskola
- **title:** esemény címe
- **chat_enabled:** hozzászólás engedélyezése
- **description:** rövid leírás
- **content:** részletes tartalom
- **user_id:** az esemény létrehozója
- **start_date:** kezdési időpont
- **end_date:** befejezési időpont
- **status:** állapot (upcoming, ongoing, ended)
- **cancelled_at:** lemondás időpontja
- **is_recurring:** ismétlődő esemény jelzése

events		
id	bigint	NN
type	events_type_enum E	NN
target_group	events_target_group_enum E	
target_class_ids	json	
target_grade_ids	json	
establishment_id	bigint	NN
title	varchar(255)	NN
chat_enabled	tinyint(1)	NN
description	text	
content	text	
user_id	bigint	NN
start_date	datetime	
end_date	datetime	
status	events_status_enum E	NN
cancelled_at	timestamp	
is_recurring	tinyint(1)	NN
recurrence_frequency	events_recurrence_frequency_enum E	
recurrence_until	datetime	
recurrence_parent_event_id	bigint	
created_at	timestamp	
updated_at	timestamp	

- **recurrence_frequency:** ismétlődés gyakorisága
- **recurrence_until:** ismétlés vége
- **recurrence_parent_event_id:** kapcsolódó fő esemény azonosítója
- **created_at, updated_at:** létrehozás és módosítás dátuma

7.8. event_shows (Esemény megjelenítés)

Megadja, hogy egy esemény mely felhasználóknál és intézményeknél jelenjen meg, illetve itt tárolódik a kedvenc jelölés és a részvételi válasz is.
Fő mezők:

- **event_id:** esemény hivatkozás
- **user_id:** felhasználó
- **establishment_id:** kapcsolódó iskola
- **is_favourite:** kedvencként jelölés
- **answer:** részvételi válasz (y / n / NA)
- **created_at, updated_at:** létrehozás és módosítás dátuma

event_shows		
id	bigint	NN
is_favourite	tinyint(1)	NN
event_id	bigint	NN
user_id	bigint	NN
answer	event_shows_answer_enum E	NN
establishment_id	bigint	NN
created_at	timestamp	
updated_at	timestamp	

7.9. event_messages (Esemény üzenetek)

Az eseményekhez kapcsolódó hozzászólásokat tárolja.
Fő mezők:

- **event_id:** esemény azonosítója
- **user_id:** az üzenet írója
- **is_important:** üzenőfali üzenet
- **content:** üzenet tartalma
- **created_at, updated_at:** létrehozás és módosítás dátuma

event_messages		
id	bigint	NN
event_id	bigint	NN
user_id	bigint	NN
is_important	tinyint(1)	NN
content	text	
created_at	timestamp	
updated_at	timestamp	

7.10. event_requests (Esemény kérések)

Azokat a kéréseket tárolja, amikor egy esemény egy adott intézményhez is eljuttatható.
Fő mezők:

- **event_id:** esemény azonosítója
- **establishment_id:** intézmény azonosítója
- **created_at, updated_at:** létrehozás és módosítás dátuma

event_requests		
id	bigint	NN
event_id	bigint	NN
establishment_id	bigint	NN
created_at	timestamp	
updated_at	timestamp	

7.11. polls (Szavazások)

Eseményhez kapcsolt szavazásokat tárolja.
Fő mezők:

- **event_id:** kapcsolódó esemény
- **title:** szavazás címe
- **user_id:** a szavazás létrehozója
- **start_date:** kezdési dátum
- **deadline:** lezárási határidő
- **is_timed:** időzített lezárás jelzése
- **hidden_results:** rejtett eredmények engedélyezése
- **is_active:** aktív állapot
- **created_at, updated_at:** létrehozás és módosítás dátuma

polls		
id	bigint	NN
event_id	bigint	NN
title	varchar(255)	
user_id	bigint	NN
start_date	date	
deadline	date	
is_timed	tinyint(1)	NN
hidden_results	tinyint(1)	NN
is_active	tinyint(1)	NN
created_at	timestamp	
updated_at	timestamp	

7.12. poll_options (Szavazási lehetőségek)

Egy szavazáshoz tartozó választlehetőségeket tárolja.
Fő mezők:

- **poll_id:** hivatkozás a szavazásra
- **title:** választlehetőség szövege
- **created_at, updated_at:** létrehozás és módosítás dátuma

poll_options		
id	bigint	NN
poll_id	bigint	NN
title	varchar(255)	
created_at	timestamp	
updated_at	timestamp	

7.13. poll_answers (Szavazatok)

A felhasználók által leadott szavazatokat tárolja.
Fő mezők:

- **poll_id:** szavazás azonosítója
- **user_id:** felhasználó azonosítója
- **poll_option_id:** kiválasztott választlehetőség

poll_answers		
poll_id	bigint	NN
user_id	bigint	NN
poll_option_id	bigint	NN

7.14. regions (Megyék)

Magyarország megyéit tárolja.
Fő mezők:

- **title:** régió neve

regions		
id	bigint	NN
title	varchar(255)	NN

7.15. inner_regions (Járások)

A megyék alá tartozó járásokat tárolja.
Fő mezők:

- **region_id:** kapcsolódó megye
- **title:** járás neve

inner_regions		
id	bigint	NN
region_id	bigint	NN
title	varchar(255)	NN

7.16. settlements (Települések)

Az egyes településeket tartalmazza, amelyekhez iskolák kapcsolódnak.
Fő mezők:

- **inner_region_id**: kapcsolódó járás
- **title:** település neve
- **number:** irányítószám

settlements		
id	bigint	NN
inner_region_id	bigint	NN
title	varchar(255)	NN
number	varchar(255)	NN

8 Tesztesetek

8.1. Frontend tesztelés

8.1.1. Sikeres regisztráció

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli az új felhasználó regisztrációját érvényes adatok megadása esetén.

A teszt során a Playwright automatizált böngésző segítségével kitöltésre kerül a regisztrációs űrlap (név, email cím, jelszó), majd az ÁSZF elfogadása után a rendszer elküldi az adatokat a backend API felé.

Sikeres regisztráció esetén a rendszer automatikusan átirányítja a felhasználót a dashboard oldalra.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test register.spec.ts –headed

A **--headed** kapcsoló biztosítja, hogy a teszt futása közben látható legyen a böngésző működése.

Név (Name)	Sikeres regisztráció
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy érvényes adatok megadása és az ÁSZF elfogadása után a rendszer sikeresen regisztrálja a felhasználót és átirányítja a dashboard oldalra.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása (http://localhost:5173/register) 2. Név mező kitöltése: Teszt Elek 3. Email mező kitöltése: teszt{timestamp}@gmail.com 4. Jelszó megadása: Teszt123 5. Jelszó megerősítése: Teszt123 6. ÁSZF, Adatvédelmi nyilatkozat checkbox bepipálása 7. Regisztráció gomb megnyomása 8. Ellenőrzés: /dashboard oldalra történő átirányítás
Tesztadatok (Test Data)	Username: Teszt Elek Email: dinamikusan generált Password: Teszt123
Várt eredmény (Expected Result)	A felhasználó sikeresen regisztrálódik és a rendszer átirányítja a /dashboard oldalra.
Tényleges eredmény (Actual Result)	A teszt sikeresen lefutott, az átirányítás megtörtént.
Típus	Funkcionális - UI Automatizált teszt

8.1.2. Regisztráció gyenge jelszóval

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi a felhasználó regisztrációját, amennyiben a megadott jelszó nem felel meg a minimális biztonsági követelményeknek (legalább 6 karakter hosszúság).

Ez a validáció fontos biztonsági szempontból, mivel a túl rövid vagy gyenge jelszavak veszélyeztethetik a felhasználói fiókok védelmét.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test register-weak-password.spec.ts –headed

A **--headed** kapcsoló lehetővé teszi a böngésző vizuális megjelenítését a teszt futása közben.

Név (Name)	Regisztráció gyenge jelszóval
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi a regisztrációt túl rövid (gyenge) jelszó esetén.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása (http://localhost:5173/register) 2. Név mező kitöltése: Teszt Elek 3. Email mező kitöltése: teszt{timestamp}@gmail.com 4. Jelszó megadása: 123 5. Jelszó megerősítése: 123 6. ÁSZF, Adatvédelmi nyilatkozat checkbox bepipálása 7. Regisztráció gomb megnyomása 8. Ellenőrzés: /dashboard oldalra történő átirányítás
Tesztadatok (Test Data)	Username: Teszt Elek Email: dinamikusan generált Password: 123
Várt eredmény (Expected Result)	A rendszer elutasítja a regisztrációt, hibaüzenetet jelenít meg, és a felhasználó a regisztrációs oldalon marad.
Tényleges eredmény (Actual Result)	A rendszer nem engedte a regisztrációt, nem történt átirányítás a /dashboard oldalra. A teszt szándékosan hibás elvárással készült (dashboard URL ellenőrzés), ezért FAIL állapotban zárult.
Típus	Funkcionális – Negatív UI Automatizált teszt

8.1.3. Regisztráció ÁSZF elfogadása nélkül

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi a felhasználó regisztrációját, amennyiben nem fogadja el az Általános Szerződési Feltételeket (ÁSZF).

Az ÁSZF elfogadása jogi és adatvédelmi szempontból kötelező feltétele a regisztrációnak, ezért a rendszernek kötelezően ellenőriznie kell a checkbox állapotát.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test register-terms-not-accepted.spec.ts --headed

A **--headed** kapcsoló lehetővé teszi a böngésző vizuális megjelenítését a teszt futása közben.

Név (Name)	Regisztráció ÁSZF elfogadása nélkül
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi a regisztrációt, ha a felhasználó nem fogadja el az ÁSZF-et és az Adatvédelmi nyilatkozatot.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása (http://localhost:5173/register) 2. Név mező kitöltése: Teszt Elek 3. Email mező kitöltése: teszt{timestamp}@gmail.com 4. Jelszó megadása: Teszt123 5. Jelszó megerősítése: Teszt123 6. ÁSZF, Adatvédelmi nyilatkozat checkbox nem kerül bepipálásra 7. Regisztráció gomb megnyomása 8. Ellenőrzés: /dashboard oldalra történő átirányítás 9. Ellenőrzés: az ÁSZF szöveg látható marad
Tesztadatok (Test Data)	Username: Teszt Elek Email: dinamikusan generált Password: Teszt123
Várt eredmény (Expected Result)	A rendszer nem engedi a regisztrációt, a felhasználó a regisztrációs oldalon marad, és az ÁSZF elfogadás hiányára utaló visszajelzés jelenik meg.
Tényleges eredmény (Actual Result)	A rendszer nem engedte a regisztrációt, nem történt átirányítás, a felhasználó a /register oldalon maradt.
Típus	Funkcionális – Negatív UI Automatizált teszt

8.1.4. Regisztráció eltérő jelszavakkal

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi a regisztrációt, amennyiben a megadott jelszó és a jelszó megerősítése mező tartalma nem egyezik meg.

Ez fontos adatbevitel-ellenőrzési (validációs) funkció, amely megakadályozza, hogy a felhasználó véletlenül hibás jelszóval hozzon létre fiókot.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test register-password-mismatch.spec.ts --headed

A **--headed** kapcsoló lehetővé teszi a teszt vizuális követését a böngészőben.

Név (Name)	Regisztráció eltérő jelszavakkal
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi a regisztrációt, ha a megadott jelszó és a jelszó megerősítése nem egyezik.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása (http://localhost:5173/register) 2. Név mező kitöltése: Teszt Elek 3. Email mező kitöltése: teszt{timestamp}@gmail.com 4. Jelszó megadása: Teszt123 5. Jelszó megerősítése: Masik123 6. ÁSZF, Adatvédelmi nyilatkozat checkbox nem kerül bepipálásra 7. Regisztráció gomb megnyomása 8. Ellenőrzés: /dashboard oldalra történő átirányítás 9. Ellenőrzés: az ÁSZF szöveg látható marad
Tesztadatok (Test Data)	Username: Teszt Elek Email: dinamikusan generált Password: Teszt123 Password confirmation: Masik123
Várt eredmény (Expected Result)	A rendszer elutasítja a regisztrációt, hibaüzenetet jelenít meg, és a felhasználó a regisztrációs oldalon marad.
Tényleges eredmény (Actual Result)	A rendszer nem engedte a regisztrációt. A teszt szándékosan hibás elvárással készült (dashboard URL ellenőrzés), ezért FAIL állapotban zárult.
Típus	Funkcionális – Negatív UI Automatizált teszt

8.1.5. Regisztráció hibás email formátummal

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi a regisztrációt, ha a felhasználó nem megfelelő formátumú email címet ad meg.

Az email formátum validáció alapvető adatellenőrzési követelmény, amely biztosítja, hogy a rendszer csak valós, szabványos formátumú email címeket fogadjon el.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test register-invalid-email.spec.ts –headed

A **--headed** opció lehetővé teszi a böngésző vizuális megjelenítését a teszt futása közben.

Név (Name)	Regisztráció hibás email formátummal
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi a regisztrációt érvénytelen email formátum esetén.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása (http://localhost:5173/register) 2. Név mező kitöltése: Teszt Elek 3. Email mező kitöltése: rosszemail 4. Jelszó megadása: Teszt123 5. Jelszó megerősítése: Teszt123 6. ÁSZF, Adatvédelmi nyilatkozat checkbox nem kerül bepipálásra 7. Regisztráció gomb megnyomása 8. Ellenőrzés: /dashboard oldalra történő átirányítás 9. Ellenőrzés: az ÁSZF szöveg látható marad
Tesztadatok (Test Data)	Username: Teszt Elek Email: rosszemail Password: Teszt123
Várt eredmény (Expected Result)	A rendszer nem engedi a regisztrációt. A böngésző validációs hiba jelenik meg, és a felhasználó a regisztrációs oldalon marad.
Tényleges eredmény (Actual Result)	A rendszer nem irányította át a felhasználót a /dashboard oldalra. A teszt szándékosan hibás elvárással készült (dashboard URL ellenőrzés), ezért FAIL állapotban zárult.
Típus	Funkcionális – Negatív UI Automatizált teszt

8.1.6. Sikeres bejelentkezés

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli a felhasználó bejelentkezését érvényes email cím és jelszó megadása esetén.

A teszt biztosítja, hogy az autentikációs folyamat (login API, token mentés, átirányítás) megfelelően működik.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test login.spec.ts –headed

A **--headed** kapcsoló segítségével a teszt futása vizuálisan követhető.

Név (Name)	Sikeres bejelentkezés
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer helyes email és jelszó megadása esetén sikeresen bejelentkezteti a felhasználót és átirányítja a dashboard oldalra.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1–2 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. LocalStorage és SessionStorage törlése (kijelentkezés szimulálása) 5. Login oldal megnyitása: (http://localhost:5173/login) 6. Email mező kitöltése a regisztrált email címmel 7. Jelszó mező kitöltése: Teszt123 8. Bejelentkezés gomb megnyomása 9. Ellenőrzés: átirányítás /dashboard oldalra
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123
Várt eredmény (Expected Result)	A rendszer sikeresen bejelentkezteti a felhasználót, és átirányítja a /dashboard oldalra.
Tényleges eredmény (Actual Result)	A rendszer sikeresen bejelentkeztette a felhasználót, és megtörtént az átirányítás a /dashboard oldalra.
Típus	Funkcionális - UI Automatizált teszt

8.1.7. Hibás bejelentkezés

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi a bejelentkezést, ha a felhasználó hibás jelszót ad meg.

Ez a teszt biztosítja, hogy az autentikációs rendszer megfelelően védi a felhasználói fiókokat, és nem enged hozzáférést érvénytelen hitelesítési adatokkal.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test login-invalid-password.spec.ts –headed

A **--headed** kapcsoló lehetővé teszi a teszt vizuális megfigyelését.

Név (Name)	Bejelentkezés hibás jelszóval
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi a bejelentkezést hibás jelszó esetén, és megfelelő hibaüzenetet jelenít meg.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1–2 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. LocalStorage és SessionStorage törlése (kijelentkezés szimulálása) 5. Login oldal megnyitása: (http://localhost:5173/login) 6. Email mező kitöltése a regisztrált email címmel 7. Jelszó mező kitöltése: RosszJelszo123 8. Bejelentkezés gomb megnyomása 9. Ellenőrzés: marad a /login oldalon 10. Ellenőrzés: hibaüzenet megjelenik
Tesztadatok (Test Data)	Email: dinamikusan generált Password: RosszJelszo123
Várt eredmény (Expected Result)	A rendszer elutasítja a bejelentkezést, a felhasználó a /login oldalon marad, és hibaüzenet jelenik meg.
Tényleges eredmény (Actual Result)	A rendszer nem engedte a bejelentkezést, nem történt dashboard átirányítás, és megjelent a hibaüzenet.
Típus	Funkcionális – Negatív UI Automatizált teszt

8.1.8. Sikeres kijelentkezés

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli a felhasználó kijelentkezését, valamint törli a hitelesítési adatokat a böngésző tárolójából.

Ez biztosítja, hogy a kijelentkezés után a felhasználó ne férhessen hozzá a védett oldalakhoz.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test logout.spec.ts –headed

A **--headed** kapcsoló biztosítja a vizuális futtatást.

Név (Name)	Sikeres kijelentkezés
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel).
Cél (Objective)	Ellenőrizni, hogy a rendszer sikeresen kijelentkezteti a felhasználót, törli a tárolt adatokat, és visszairányítja a főoldalra.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	1–2 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. Felhasználói avatar ikon megnyitása 5. Kijelentkezés gomb megjelenésének ellenőrzése 6. Kijelentkezés gomb megnyomása 7. Ellenőrzés: átirányítás a főoldalra (/) 8. Ellenőrzés: localStorage-ben az esemenyter_user érték törlésre került
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123
Várt eredmény (Expected Result)	A rendszer átirányít a főoldalra (/) A localStorage-ből törlődik az esemenyter_user adat A felhasználó kijelentkezett állapotba kerül
Tényleges eredmény (Actual Result)	A rendszer sikeresen kijelentkeztette a felhasználót, megtörtént az átirányítás a főoldalra, és a tárolt user adat törlésre került.
Típus	Funkcionális – UI Automatizált teszt

8.1.9. Diák profil beállítás sikeresen

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli a diák szerepkör kiválasztását és a profilbeállítási folyamatot (setup wizard).

A teszt biztosítja, hogy a felhasználó végig tud haladni a régió, település, intézmény kiválasztási folyamaton, majd sikeresen mentheti a profilját.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test student_setup.spec.ts –headed

A **--headed** kapcsoló biztosítja a böngésző vizuális megjelenítését.

Név (Name)	Diák profil beállítás sikeresen
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel). A kiválasztott régió, város és intézmény létezik az adatbázisban A felhasználó még nem rendelkezik szerepkörrel
Cél (Objective)	Ellenőrizni, hogy a felhasználó diák szerepkör kiválasztása után végig tudja vinni a setup varázslót, és a rendszer a "pending approval" oldalra irányítja.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	2–3 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. Diák szerepkör kiválasztása 5. Setup wizard elindítása 6. Régió kiválasztása (pl. Bács-Kiskun) 7. Járás kiválasztása (pl. Kiskunfélegyházai járás) 8. Település kiválasztása (pl. Kiskunfélegyháza) 9. Intézmény kiválasztása 10. Profil mentése 11. Ellenőrzés: átirányítás /pending-approval oldalra
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123 Régió: Bács-Kiskun Járás: Kiskunfélegyházi Város: Kiskunfélegyháza Intézmény: PÉGÉ
Várt eredmény (Expected Result)	A rendszer sikeresen elmenti a diák profil beállítását, és a felhasználót a /pending-approval oldalra irányítja.
Tényleges eredmény (Actual Result)	A setup folyamat sikeresen lefutott, és a rendszer átirányította a felhasználót a /pending-approval oldalra.
Típus	Funkcionális – End-to-End UI Automatizált teszt

8.1.10. Tanár profil beállítás sikeresen

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli a tanár szerepkör kiválasztását és a profilbeállítási folyamatot.

A teszt biztosítja, hogy a felhasználó végig tud haladni a régió, település és intézmény kiválasztási folyamaton, majd sikeresen mentheti a profilját tanárként.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test teacher_setup.spec.ts –headed

A **--headed** opció lehetővé teszi a teszt vizuális ellenőrzését.

Név (Name)	Tanár profil beállítás sikeresen
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel). A kiválasztott régió, város és intézmény létezik az adatbázisban A felhasználó még nem rendelkezik szerepkörrel
Cél (Objective)	Ellenőrizni, hogy a felhasználó tanár szerepkör kiválasztása után sikeresen végig tudja vinni a setup varázslót, és a rendszer a „pending approval" oldalra irányítja.
Prioritás (Priority)	Magas
Becsült idő (Estimated Time)	2–3 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. Tanár szerepkör kiválasztása 5. Setup wizard elindítása 6. Régió kiválasztása (pl. Bács-Kiskun) 7. Járás kiválasztása (pl. Kiskunfélegyházai járás) 8. Település kiválasztása (pl. Kiskunfélegyháza) 9. Intézmény kiválasztása 10. Profil mentése 11. Ellenőrzés: átirányítás /pending-approval oldalra
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123 Régió: Bács-Kiskun Járás: Kiskunfélegyházi Város: Kiskunfélegyháza Intézmény: PÉGÉ
Várt eredmény (Expected Result)	A rendszer sikeresen elmenti a tanár profil beállítását, és a felhasználót a /pending-approval oldalra irányítja.
Tényleges eredmény (Actual Result)	A setup folyamat sikeresen lefutott, és a rendszer átirányította a felhasználót a /pending-approval oldalra.
Típus	Funkcionális – End-to-End UI Automatizált teszt

8.1.11. Admin intézmény létrehozása sikeresen

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelően kezeli az admin szerepkörű felhasználó intézmény létrehozási folyamatát.

Ez a teszt biztosítja, hogy az admin végig tud haladni a több lépésből álló varázslón (wizard), meg tudja adni az intézmény adatait, és sikeresen létrejön az intézmény profil.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test admin_school-profile.spec.ts --headed

A **--headed** opció lehetővé teszi a vizuális ellenőrzést.

Név (Name)	Admin intézmény létrehozása sikeresen
Allapot (Status)	Kész
Elöfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel). A kiválasztott régió, város és intézmény létezik az adatbázisban Az intézmény neve még nem létezik az adatbázisban A felhasználó még nem rendelkezik szerepkörrel
Cél (Objective)	Ellenörizni, hogy az admin szerepkört választó felhasználó sikeresen végig tudja vinni az intézmény létrehozási folyamatot, és a rendszer az intézmény dashboard oldalra irányitja.
Prioritás (Priority)	Nagyon magas
Becsült idö (Estimated Time)	3–5 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenörzés: átirányitás /dashboard oldalra 4. Token meglétének ellenörzése localStorage-ben 5. Admin szerepkör kiválasztása 6. Setup wizard elinditása 7. Régió kiválasztása (pl. Bács-Kiskun) 8. Járás kiválasztása (pl. Kiskunfélegyházai járás) 9. Település kiválasztása (pl. Kiskunfélegyháza) 10. Intézményi adatok ürlap megjelenésének ellenörzése 11. Intézmény adatainak kitöltése: - Név (dinamikus, timestamp alapú) - Leirás - Cim - Telefonszám - Email - Weboldal 12. Tovább gomb megnyomása 13. Admin nyilatkozat checkbox bepipálása 14. Véglegesités 15. Ellenörzés: átirányitás /institution-dashboard oldalra
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123 Intézmény név: dinamikusan generált Leirás: Ez egy teszt intézmény Cim: Teszt utca 1. Telefon: +36 1 234 5678 Email: dinamikusan generált Web: https://www.tesztiskola.hu
Várt eredmény (Expected Result)	A rendszer sikeresen létrehozza az intézményt, és az admin felhasználót átirányitja az /institution-dashboard oldalra.
Tényleges eredmény (Actual Result)	Az intézmény sikeresen létrejött, és a rendszer átirányitotta a felhasználót az /institution-dashboard oldalra.
Tipus	Funkcionális – End-to-End UI Automatizált teszt Komplex szerepkör alapú folyamat teszt

8.1.12. Admin intézmény létrehozása duplikált névvel (hibakezelés)

A teszteset célja annak ellenőrzése, hogy a rendszer nem engedi új intézmény létrehozását, ha az adott intézménynév már létezik az adatbázisban.

Ez a teszt biztosítja az adatbázis szintű egyediség (unique constraint) és a backend validáció megfelelő működését.

Teszt futtatása:
A frontend teszt futtatásához a következő parancs szükséges:

npx playwright test admin_school-profile-duplicate.spec.ts --headed

A **--headed** opció biztosítja a vizuális futtatást.

Név (Name)	Admin intézmény létrehozása duplikált névvel
Állapot (Status)	Kész
Előfeltétel (Precondition)	A frontend és backend fut (Vue + Laravel). Az adatbázisban már létezik egy intézmény az adott névvel (pl. PÉGÉ) A felhasználó még nem rendelkezik szerepkörrel
Cél (Objective)	Ellenőrizni, hogy a rendszer nem engedi új intézmény létrehozását, ha a megadott intézménynév már létezik az adatbázisban.
Prioritás (Priority)	Nagyon magas
Becsült idő (Estimated Time)	3–5 perc
Teszt szkript lépések (Steps)	1. Regisztrációs oldal megnyitása 2. Új felhasználó regisztrálása érvényes adatokkal 3. Ellenőrzés: átirányítás /dashboard oldalra 4. Token meglétének ellenőrzése 5. Admin szerepkör kiválasztása 6. Setup wizard elindítása 7. Régió kiválasztása (pl. Bács-Kiskun) 8. Járás kiválasztása (pl. Kiskunfélegyházai járás) 9. Település kiválasztása (pl. Kiskunfélegyháza) 10. Intézményi adatok űrlap megjelenésének ellenőrzése 11. Intézmény név megadása: PÉGÉ (már létező név) 12. További mezők kitöltése 13. Tovább gomb megnyomása 14. Admin nyilatkozat checkbox bepipálása 15. Véglegesítés 16. Ellenőrzés: a rendszer nem irányít át az /institution-dashboard oldalra
Tesztadatok (Test Data)	Email: dinamikusan generált Password: Teszt123 Intézmény név: PÉGÉ (létező név) Leírás: Ez egy teszt intézmény Cím: Teszt utca 1. Telefon: +36 1 234 5678 Email: dinamikusan generált Web: https://www.tesztiskola.hu
Várt eredmény (Expected Result)	A rendszer elutasítja az intézmény létrehozását, és nem történik átirányítás az /institution-dashboard oldalra. Hibaüzenet jelenik meg a duplikált név miatt.
Tényleges eredmény (Actual Result)	A rendszer nem hozta létre az intézményt. Nem történt átirányítás az /institution-dashboard oldalra. A teszt szándékosan dashboard URL ellenőrzést tartalmaz, ezért FAIL állapotban zárul.
Típus	Funkcionális – Negatív End-to-End UI Automatizált teszt Adatbázis egyediség (unique constraint) ellenőrzés

8.2. Backend tesztelés

8.2.1. Regisztráció – sikeres

A teszteset célja annak ellenőrzése, hogy a rendszer érvényes adatokkal sikeresen létrehozza az új felhasználót, és visszaadja a szükséges autentikációs adatokat.

Ez a teszt biztosítja a regisztrációs folyamat helyes működését és az alapvető mezők backend validációját.

Teszt futtatása:

POST /api/register hívás érvényes adatokkal (egyedi email cím).

Elvárt eredmény: 200, sikeres válasz, user és token mezők visszaadása.

Mező	Leírás
Név (Name)	Regisztráció – sikeres
Állapot (Status)	ready
Előfeltétel (Precondition)	Az email cím még ne létezzen az adatbázisban.
Cél (Objective)	A POST /api/register végpont helyes működésének ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) POST /api/register hívás küldése
Hiba típusa (Issue Type)	validation
Test Script Test - Tesztadatok (Test Data)	"username":"John", "email":"john.unique@example.com", "password":"Password123", "password_confirmation":"Password123"
Test Script Test - Várt eredmény (Expected Result)	200; message, user, token
Test Script Test - Tényleges eredmény (Actual Result):	200; message, user, tokenk
leírás	Új felhasználó létrehozása érvényes adatokkal.

8.2.2. Login – hibás jelszó

A teszteset célja annak ellenőrzése, hogy hibás jelszó megadása esetén a rendszer megtagadja a bejelentkezést.

Ez a teszt biztosítja a hitelesítés negatív ágának és a hibakezelésnek a megfelelő működését.

Teszt futtatása:

POST /api/login hívás létező emaillel és hibás jelszóval.

Elvárt eredmény: 401, megfelelő hibaüzenet.

Mező	Leírás
Név (Name)	Login – hibás jelszó
Állapot (Status)	ready
Előfeltétel (Precondition)	Létező user az adatbázisban.
Cél (Objective)	POST /api/login negatív ág ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) POST /api/login hívás
Hiba típusa (Issue Type)	permission
Test Script Test - Tesztadatok (Test Data)	"email":"test@example.com", "password":"WrongPass123"
Test Script Test - Várt eredmény (Expected Result)	401; hibaüzenet (Hibás jelszó!)
Test Script Test - Tényleges eredmény (Actual Result):	401; hibaüzenet (Hibás jelszó!)
leírás	Létező email + rossz jelszó esetén elutasítás.

8.2.3. Logout – token nélkül

A teszteset célja annak ellenőrzése, hogy a védett kijelentkezési végpont token nélkül nem érhető el.

Ez a teszt biztosítja az auth middleware helyes működését.

Teszt futtatása:

DELETE /api/logout Authorization fejléc nélkül.

Elvárt eredmény: 401.

Mező	Leírás
Név (Name)	Logout – token nélkül
Állapot (Status)	ready
Előfeltétel (Precondition)	Nincs Bearer token fejléce.
Cél (Objective)	DELETE /api/logout auth védelmének ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) DELETE /api/logout token nélkül
Hiba típusa (Issue Type)	permission
Test Script Test - Tesztadatok (Test Data)	Header: nincs Authorization
Test Script Test - Várt eredmény (Expected Result)	401;
Test Script Test - Tényleges eredmény (Actual Result):	401;
leírás	Auth védett végpont hívása token nélkül.

8.2.4. Régió keresés

A teszteset célja annak ellenőrzése, hogy a régió kereső végpont megfelelően szűr a query paraméter alapján.

Ez a teszt biztosítja a lista-visszaadás, keresés és tartalmi helyesség működését.

Teszt futtatása:

GET /api/regions?search=B.

Elvárt eredmény: 200, releváns, rendezett találati lista.

Mező	Leírás
Név (Name)	Régió keresés
Állapot (Status)	ready
Előfeltétel (Precondition)	Seedelt régió adatok elérhetők.
Cél (Objective)	GET /api/regions?search=B eredményének ellenőrzése.
Prioritás (Priority)	Medium
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) GET /api/regions?search=B hívás
Hiba típusa (Issue Type)	content
Test Script Test - Tesztadatok (Test Data)	search=B
Test Script Test - Várt eredmény (Expected Result)	200; data lista, rendezett elemek
Test Script Test - Tényleges eredmény (Actual Result):	200; data lista, rendezett elemek
leírás	Keresett régiólista visszaadása query param alapján.

8.2.5. Intézmény létrehozás – sikeres

A teszteset célja annak ellenőrzése, hogy admin jogosultsággal és érvényes adatokkal új intézmény hozható létre.

Ez a teszt biztosítja az intézmény létrehozási folyamat és az adatok mentésének helyes működését.

Teszt futtatása:

POST /api/establishment/create érvényes Bearer tokennel és kötelező mezőkkel.

Elvárt eredmény: 201, sikeres válasz, létrehozott intézmény azonosítóval.

Mező	Leírás
Név (Name)	Intézmény létrehozás – sikeres
Állapot (Status)	ready
Előfeltétel (Precondition)	Érvényes Bearer token, létező settlement_id.
Cél (Objective)	GET /api/regions?search=B eredményének ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) POST /api/establishment/create hívás tokennel
Hiba típusa (Issue Type)	integration
Test Script Test - Tesztadatok (Test Data)	"title":"My School2", "description":"Short description", "settlement_id":2
Test Script Test - Várt eredmény (Expected Result)	201; message, data.id
Test Script Test - Tényleges eredmény (Actual Result):	201; message, data.id
leírás	Új intézmény mentése auth userrel.

8.2.6. Osztály létrehozás – hiányzó név

A teszteset célja annak ellenőrzése, hogy kötelező mező hiánya esetén a rendszer validációs hibát ad.

Ez a teszt biztosítja a backend input validáció megfelelő működését.

Teszt futtatása:

POST /api/establishment/classes/create name mező nélkül.

Elvárt eredmény: 422, validációs hiba.

Mező	Leírás
Név (Name)	Osztály létrehozás – hiányzó név
Állapot (Status)	ready
Előfeltétel (Precondition)	Érvényes token.
Cél (Objective)	POST /api/establishment/classes/create validációjának ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) Hívás name nélkül
Hiba típusa (Issue Type)	validation
Test Script Test - Tesztadatok (Test Data)	"grade":10, "establishment_id":1
Test Script Test - Várt eredmény (Expected Result)	422; validációs hiba
Test Script Test - Tényleges eredmény (Actual Result):	422; validációs hiba
leírás	Kötelező mező hiánya esetén validációs hiba.

8.2.7. Kérelmek kezelése – elfogadás

A teszteset célja annak ellenőrzése, hogy admin jogosultság mellett a rendszer képes több intézményi kérelmet elfogadni.

Ez a teszt biztosítja a kérelmek tömeges feldolgozásának helyes működését.

Teszt futtatása:

POST /api/establishment/requests/handle admin tokennel, action=accept, request_id lista megadásával.

Elvárt eredmény: 200, accepted számláló visszaadása.

Mező	Leírás
Név (Name)	Kérelmek kezelése – elfogadás
Állapot (Status)	ready
Előfeltétel (Precondition)	Admin jogosultságú token, létező request ID-k.
Cél (Objective)	POST /api/establishment/requests/handle működésének ellenőrzése
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) Kérelmek elfogadása
Hiba típusa (Issue Type)	integration
Test Script Test - Tesztadatok (Test Data)	"establishment_id":1, "action":"accept", "request_id":[1,2]
Test Script Test - Várt eredmény (Expected Result)	200; válaszban accepted számláló
Test Script Test - Tényleges eredmény (Actual Result):	200; válaszban accepted számláló
leírás	Több kérelem elfogadása egy hívásban.

8.2.8. Kérelmek kezelése – nem admin felhasználó

A teszteset célja annak ellenőrzése, hogy nem admin felhasználó nem kezelhet intézményi kérelmeket.

Ez a teszt biztosítja a jogosultságkezelés helyes működését.

Teszt futtatása:

POST /api/establishment/requests/handle nem admin tokennel.

Elvárt eredmény: 403, jogosultsági hibaüzenet.

Mező	Leírás
Név (Name)	Kérelmek kezelése – nem admin felhasználó
Állapot (Status)	ready
Előfeltétel (Precondition)	Érvényes token, de a user nem admin az adott intézményben.
Cél (Objective)	POST /api/establishment/requests/handle jogosultságkezelés ellenőrzése.
Prioritás (Priority)	High
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) Kérelmek kezelése nem admin tokennel
Hiba típusa (Issue Type)	permission
Test Script Test - Tesztadatok (Test Data)	"establishment_id":1, "action":"accept", "request_id":[1]
Test Script Test - Várt eredmény (Expected Result)	403, Nem Felhatalmazott!
Test Script Test - Tényleges eredmény (Actual Result):	403, Nem Felhatalmazott!
leírás	Nem admin user próbál kérelmeket feldolgozni.

8.2.9. Diáklista – nem létező intézmény

A teszteset célja annak ellenőrzése, hogy a rendszer megfelelő hibát ad vissza nem létező intézmény azonosító esetén.

Ez a teszt biztosítja a hibás erőforrás-hivatkozások kezelését.

Teszt futtatása:

GET /api/members/students/{establishment} nem létező azonosítóval.

Elvárt eredmény: 400 vagy 404 (a specifikációtok szerint), megfelelő hibaüzenet.

Mező	Leírás
Név (Name)	Diáklista – nem létező intézmény
Állapot (Status)	ready
Előfeltétel (Precondition)	Olyan establishment_id, ami biztosan nem létezik.
Cél (Objective)	GET /api/members/students/{establishment} hibakezelésének ellenőrzése.
Prioritás (Priority)	Medium
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) GET /api/members/students/999999 hívás
Hiba típusa (Issue Type)	content
Test Script Test - Tesztadatok (Test Data)	establishment_id=999999
Test Script Test - Várt eredmény (Expected Result)	400, hibaüzenet (Intézmény nem található!)
Test Script Test - Tényleges eredmény (Actual Result):	400, hibaüzenet (Intézmény nem található!)
leírás	Publikus végpont hívása hibás intézmény azonosítóval.

8.2.10. Esemény létrehozás – hibás dátumtartomány

A teszteset célja annak ellenőrzése, hogy a rendszer elutasítja azt az eseményt, ahol az end_date korábbi, mint a start_date.

Ez a teszt biztosítja az időintervallum validáció és üzleti szabályok helyes működését.

Teszt futtatása:

POST /api/events érvényes tokennel, hibás dátumtartománnyal.

Elvárt eredmény: 422, end_date mezőre vonatkozó validációs hiba.

Mező	Leírás
Név (Name)	Esemény létrehozás – hibás dátumtartomány
Állapot (Status)	ready
Előfeltétel (Precondition)	Érvényes token.
Cél (Objective)	POST /api/events dátumvalidáció ellenőrzése.
Prioritás (Priority)	Medium
Felelős (Owner)	Tóth Tamás
Teszt szkript lépések (Steps):	1) Hibás dátummal esemény létrehozása
Hiba típusa (Issue Type)	validation
Test Script Test - Tesztadatok (Test Data)	"type":"global", "title":"Test", "description":"Test", "start_date":"2026-03-10 10:00:00", "end_date":"2026-03-10 09:00:00"
Test Script Test - Várt eredmény (Expected Result)	422; validációs hiba (end_date)
Test Script Test - Tényleges eredmény (Actual Result):	422; validációs hiba (end_date)
leírás	end_date korábbi, mint start_date.

9 Felhasználói dokumentáció

9.1. A rendszer rövid bemutatása

Az EseményTér egy webes felület, amely az iskolai események szervezését és kommunikációját támogatja. A rendszerben a felhasználók szerepkör alapján férnek hozzá az egyes funkciókhoz.

Támogatott szerepkörök:

- vendég (nem bejelentkezett felhasználó)
- diák
- tanár
- adminisztrátor

9.2. Belépés és fiókhasználat

9.2.1. Regisztráció

1. Nyisd meg a Regisztráció oldalt.
2. Add meg a nevedet, email címedet és jelszavadat.
3. Erősítsd meg a jelszót.
4. Fogadd el az ÁSZF-et és az adatvédelmi nyilatkozatot.
5. Kattints a Regisztráció gombra.

9.2.2. Email megerősítés

1. A rendszer megerősítő emailt küld a megadott címre.
2. Az emailben található linkre kattintva hitelesítsd a fiókot.
3. Sikeres megerősítés után be tudsz jelentkezni és teljes hozzáférést kapsz.

9.2.3. Bejelentkezés

1. A Bejelentkezés oldalon add meg email címedet és jelszavadat.
2. Igény szerint használd az Emlékezz rám opciót.
3. Sikeres bejelentkezés után a rendszer a megfelelő irányítópultra navigál.

9.2.4. Bejelentkezés szerepkör szerint (diák, tanár, admin)

A regisztráció után a felhasználó kiválasztja, milyen szerepkörben szeretné használni a rendszert.

Diák jelentkezés:

1. Válaszd a diák szerepkört a profilbeállítás során.
2. Add meg az intézményi adatokat (régió, település, intézmény).
3. Küldd be az intézményi csatlakozási kérelmet diák szerepkörrel.
4. Az intézményi admin jóváhagyása után aktiválódik a diák tagság.

Tanár jelentkezés:

1. Válaszd a tanár szerepkört a profilbeállítás során.
2. Add meg az intézményi adatokat (régió, település, intézmény).
3. Küldd be az intézményi csatlakozási kérelmet tanár szerepkörrel.
4. Jóváhagyás után a rendszer tanári jogosultságokat biztosít.

Admin jelentkezés:

1. Válaszd az admin/intézményvezetői irányt a profilbeállítás során.
2. Add meg az intézményhez tartozó kötelező adatokat.
3. Hozd létre az intézményi profilt.
4. Sikeres mentés után elérhetővé válik az intézményvezetői irányítópult.

9.3. Navigáció és fő oldalak

Bejelentkezés után a fő menüből elérhetők:

- Felhasználói irányítópult
- Események listája
- Eseménynaptár
- Profil
- Kijelentkezés

A rendszer reszponzív, ezért asztali gépen és mobil eszközön is használható.

9.4. Események kezelése felhasználóként

9.4.1. Eseménylista

Az Események oldalon a felhasználó:

- áttekintheti a számára elérhető eseményeket
- gyors összesítést lát az aktív eseményekről
- naptár nézetbe válthat

9.4.2. Esemény részletek

Az esemény adatlapja tartalmazza:

- esemény címét és leírását
- típusát és állapotát
- szervezői adatokat
- időpontokat
- felhasználói műveleteket (pl. jelentkezés, kedvencek, megosztás)

9.4.3. Részvételi visszajelzés

A felhasználó az esemény adatlapján tud visszajelzést adni arról, hogy részt vesz-e az eseményen.

9.4.4. Kedvenc jelölés

Az esemény kedvencként jelölhető, illetve a jelölés visszavonható.

9.4.5. Kommentelés

Bejelentkezett felhasználók hozzászólást írhatnak az eseményekhez. A rendszer a túl hosszú kommentet nem engedi menteni.

9.5. Eseménynaptár használata

A Naptár nézetben a felhasználó:

- havi bontásban látja az eseményeket
- válthat az előző és a következő hónapok között
- a napi mezőkben rövid eseménylistát kap
- egy eseményre kattintva megnyithatja annak részletes oldalát

Az ismétlődő események alkalmai és a törölt alkalmak jelölése is megjelenik.

9.6. Esemény létrehozása

Az esemény létrehozás funkció jogosultsághoz kötött.

A folyamat fő lépései:

1. Esemény szintjének kiválasztása (iskolai vagy globális).
2. Célcsoport vagy érintett terület megadása.
3. Esemény alapadatainak kitöltése.
4. Időzítés és opcionális ismétlődés beállítása.
5. Mentés és közzététel.

Ha a felhasználó nem jogosult eseményt létrehozni, a rendszer figyelmeztető üzenetet jelenít meg.

9.7. Profilkezelés

A Profil oldalon a felhasználó:

- megtekintheti saját adatait
- szerkesztheti a profilinformációit
- ellenőrizheti aktuális szerepkörét

9.8. Intézményvezetői funkciók

Az intézményvezetői irányítópult az alábbi funkciókat tartalmazza:

- intézményi statisztikák megtekintése
- globális esemény meghívások elfogadása vagy elutasítása
- tanári és diák csatlakozási kérelmek kezelése
- kérelmek keresése és tömeges feldolgozása

9.9. Függő jóváhagyás állapot

Ha a felhasználó intézményi kérelem elbírálására vár, a rendszer külön oldalon jelzi, hogy a kérelem feldolgozás alatt áll.

9.10. Jogszabályi tájékoztatók

A rendszerben külön oldalon érhető el:

- Általános Szerződési Feltételek (ÁSZF)
- Adatvédelmi nyilatkozat

9.11. Rövid felhasználói folyamatok

9.11.1. Diák

1. Regisztráció és email megerősítés.
2. Bejelentkezés.
3. Események böngészése.
4. Részvételi visszajelzés és kommentelés.
5. Naptár rendszeres követése.

9.11.2. Tanár

1. Bejelentkezés.
2. Események kezelése és kommunikáció.
3. Jogosultság esetén esemény létrehozása.
4. Naptár alapú követés.

9.11.3. Adminisztrátor

1. Intézményvezetői irányítópult használata.
2. Csatlakozási kérelmek kezelése.
3. Globális meghívások feldolgozása.
4. Intézményi események felügyelete.